

Actividad 2

UD4

Pavel Miron

Índice

Contenido

Capturas de pantalla y explicaciones sobre el desarrollo en cada plataforma y en especial acerca de la orquestación de contenedores.....	2
Por principio	2
Preparación	2
Comprobación final	4
Responde brevemente a las siguientes pregunta:	5

Capturas de pantalla y explicaciones sobre el desarrollo en cada plataforma y en especial acerca de la orquestación de contenedores

Por principio tenemos que organizar nuestra estructura de carpetas y archivos para que el Docker Compose funcione de manera correcta, más abajo represento la estructura de carpetas, además voy a utilizar el proyecto de una actividad anterior de Angular que tiene Front Back y Base de datos ya preparado para esta práctica con Compose:

- proyecto-root
 - frontend-angular
 - Dockerfile
 - (archivos de angular)
 - Backend-springboot
 - Dockerfile
 - (archivos de springboot)
 - docker-compose.yml

Preparación

dockerfile de back explicación:

```
FROM eclipse-temurin:21-jdk-alpine
```

Usa la versión ligera de java 21 para que el contenedor no ocupe mucho espacio.

```
COPY build/libs/*.jar app.jar
```

Copia mi archivo .jar (el ejecutable de Spring) dentro de contenedor.

```
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

El "ENTRYPOINT" hace que en cuanto el contenedor se encienda la aplicación se ejecute automáticamente.

dockerfile de front explicación:

```
FROM node:latest AS build
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN npm install && npm run build
```

Esta es primera etapa -> uso **Node.js** solo para instalar las librerías y "traducir" mi código Angular a archivos que cualquier navegador entienda (HTML y JS).

```
FROM nginx:alpine
COPY --from=build /app/dist/*/browser
/usr/share/nginx/html
EXPOSE 80
```

Segunda etapa -> Como no necesito Node para que la web funcione, paso esos archivos limpios a un servidor **Nginx**, que es mucho más rápido y ligero. Así, el contenedor final ocupa poquísimo espacio.

En docker-compose.yml

```
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_DATABASE: dcuniverse
      MYSQL_ROOT_PASSWORD: daw12
    volumes:
      - data_db:/var/lib/mysql

  backend:
    build: ./backend-springboot
    ports:
      - "8080:8080"
    depends_on:
      - db

  frontend:
    build: ./frontend-angular
    ports:
      - "4200:80"
    depends_on:
      - backend

volumes:
  data_db:
```

Bloque db:

db: en este bloque lo que hago es descargar la imagen oficial de MySQL versión 8.0.

enviroment: aquí introducimos el nombre de base de datos y la contraseña con la cual puede acceder.

volumens: es la parte de **persistencia**, donde decidimos que todo que se guarde en la carpeta interna de MySQL. También guardo en volumen **data_db** porque asísi borramos el contenedor los datos no se borran.

Bloque Backend:

build: no descarga una imagen de internet si no lo utiliza el dockerfile que lo hemos utilizado/creado en back con springboot

ports: utilizar básicamente el puerto 8080 para que pueda entrar desde navegador a la API.

depends_on: es importante porque dice al backend que se espere a que la BD este encendida antes de intentar arrancar.

Bloque Frontend:

build: igual como en el bloque del back.

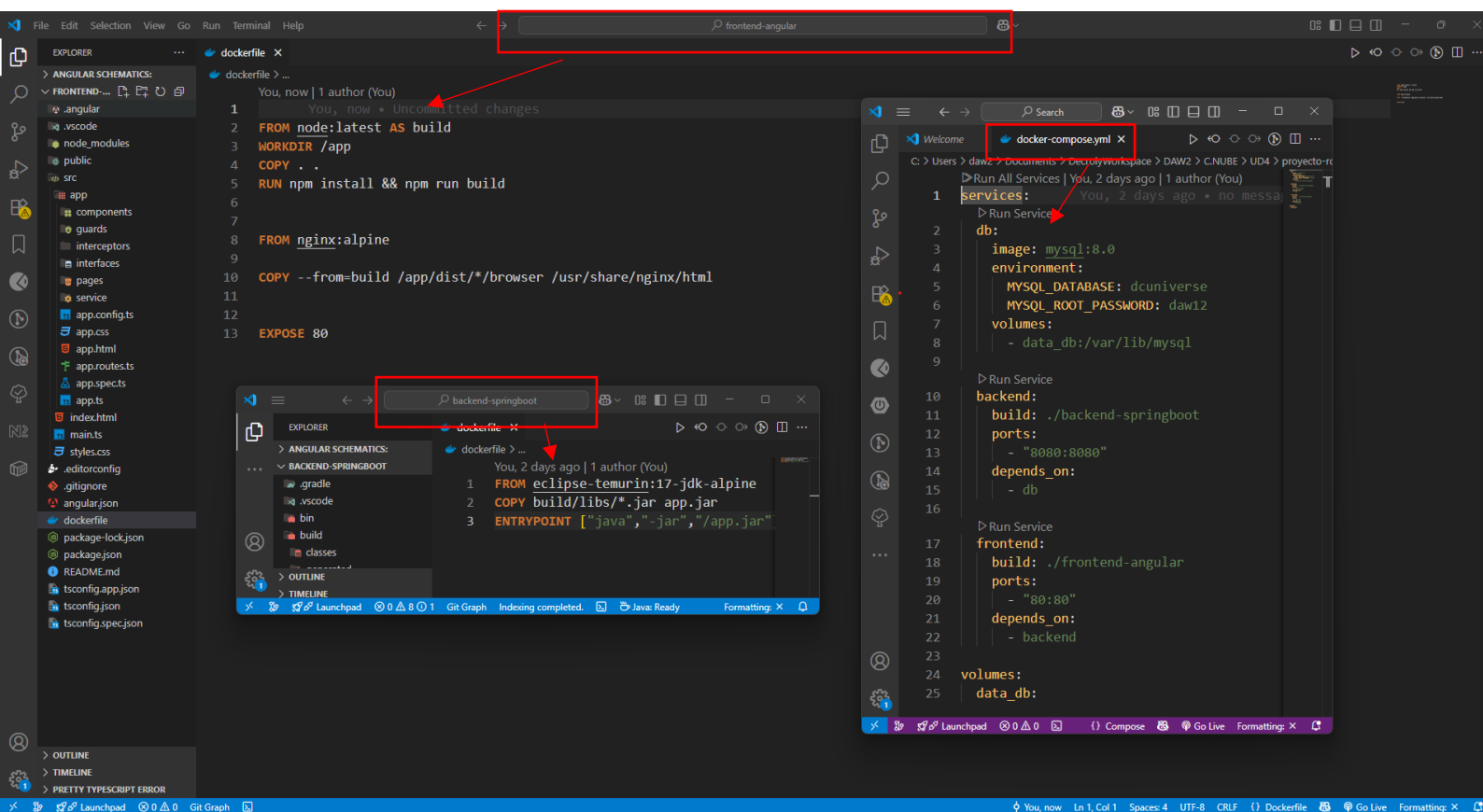
ports: como es una web usa el puerto estándar, al utilizar localhost accede aquí.

depends_on: se asegura de que la API este lista antes de que la web intente pedir datos.

Bloque Volumens:

data_db: aquí es donde declaramos oficialmente el volumen, es como crear un disco duro virtual en Docker.

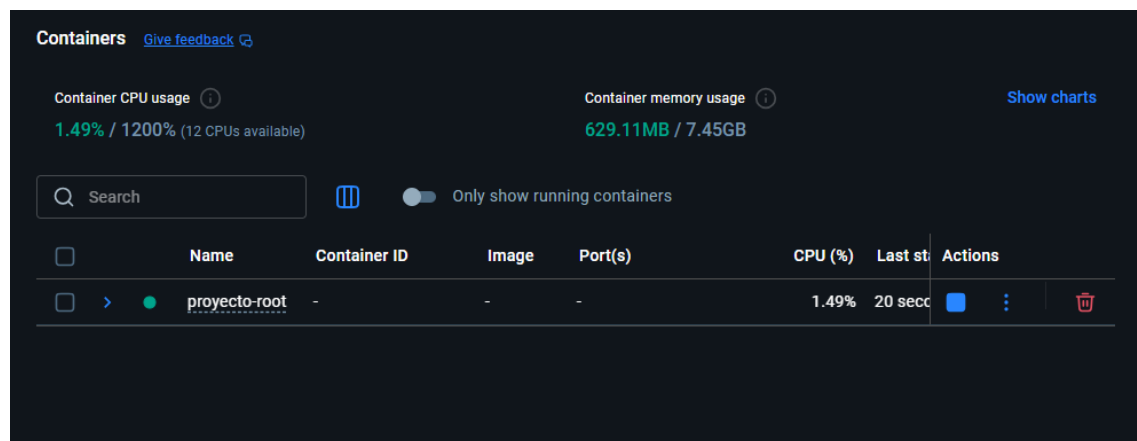
¡Para comprobar que el docker-compose.yml esta bien escrito podemos ejecutar “docker compose config”, en el terminal del mismo con VSCode.!



Al final los tres ficheros tienen que quedar así.

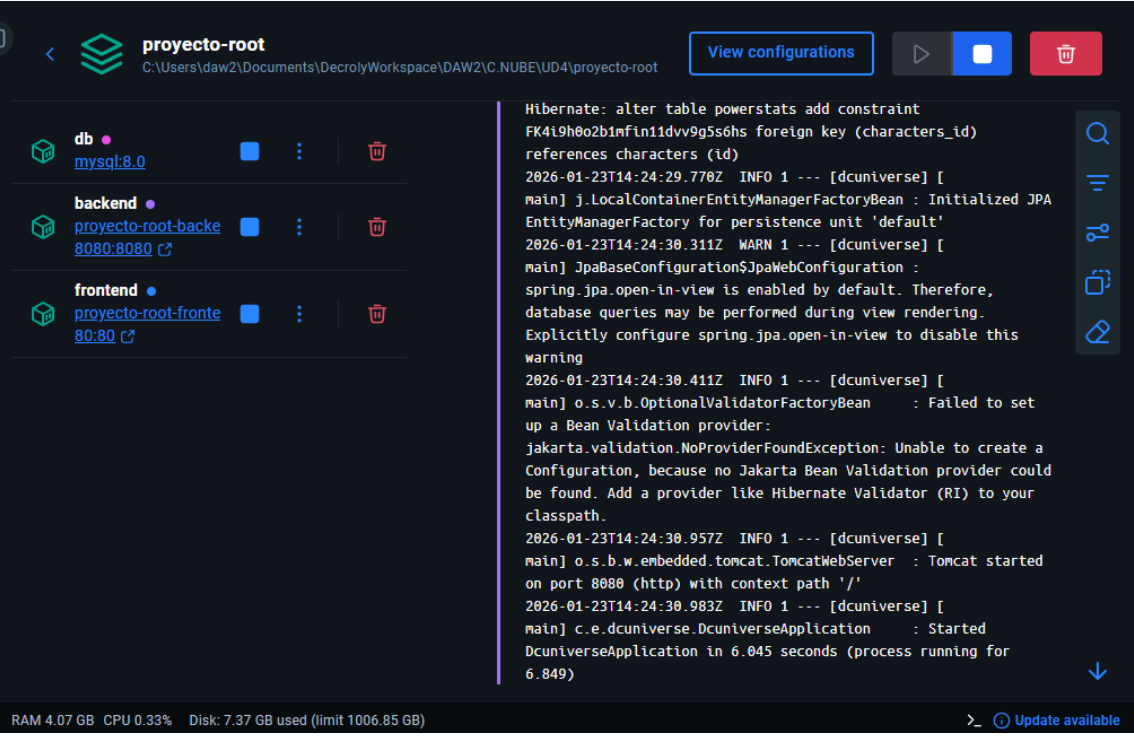
Comprobación final

Ejecutamos el comando `docker-compose up --build` dentro de nuestro proyecto donde tenemos el `docker-compose.yml`, que este comando es levantar/crear todo el stack que hemos creado a la vez, así mismo creamos el contenedor que lo podemos ver dentro de Docker Desktop. También quiero asegurarme de que cualquier cambio realizado en el código de Angular o Spring Boot se viera reflejado al instante, recreando las imágenes desde cero antes de levantar los contenedores y los volúmenes de persistencia.



Al crearse se levanta automáticamente.

Al pulsar en el contenedor podemos ver todas tus partes si cumplen el funcionamiento, junto con los logs que enseña los errores/movimientos.



The screenshot shows the Docker Desktop interface for a container named 'proyecto-root'. The left sidebar lists three containers: 'db' (mysql:8.0), 'backend' (proyecto-root-backe:8080:8080), and 'frontend' (proyecto-root-fronte:80:80). The main panel displays the logs for the 'proyecto-root' container. The logs show the application starting, initializing JPA, and encountering a warning about 'spring.jpa.open-in-view'. A critical error occurs: 'jakarta.validation.NoProviderFoundException: Unable to create a Configuration, because no Jakarta Bean Validation provider could be found. Add a provider like Hibernate Validator (RI) to your classpath.' The application then starts successfully on port 8080.

```
Hibernate: alter table powerstats add constraint
FK4i9h0o2b1mf1n11dv9g5s6hs foreign key (characters_id)
references characters (id)
2026-01-23T14:24:29.770Z INFO 1 --- [dcuniverse] [
main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA
EntityManagerFactory for persistence unit 'default'
2026-01-23T14:24:30.311Z WARN 1 --- [dcuniverse] [
main] JpaBaseConfiguration$JpaWebConfiguration :
spring.jpa.open-in-view is enabled by default. Therefore,
database queries may be performed during view rendering.
Explicitly configure spring.jpa.open-in-view to disable this
warning
2026-01-23T14:24:30.411Z INFO 1 --- [dcuniverse] [
main] o.s.v.b.OptionalValidatorFactoryBean : Failed to set
up a Bean Validation provider:
jakarta.validation.NoProviderFoundException: Unable to create a
Configuration, because no Jakarta Bean Validation provider could
be found. Add a provider like Hibernate Validator (RI) to your
classpath.
2026-01-23T14:24:30.957Z INFO 1 --- [dcuniverse] [
main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started
on port 8080 (http) with context path '/'
2026-01-23T14:24:30.983Z INFO 1 --- [dcuniverse] [
main] c.e.dcuniverse.DcuniverseApplication : Started
DcuniverseApplication in 6.045 seconds (process running for
6.849)
```

RAM 4.07 GB CPU 0.33% Disk: 7.37 GB used (limit 1006.85 GB) >_ Update available

Desde este momento, tenemos la posibilidad de arrancar el Compose que tiene dentro la base de datos, back y front a la vez, sin necesidad de ejecutar cada una aparte.

Responde brevemente a las siguientes preguntas:

- Utilizando tus palabras, explica las diferencias entre Docker Compose y Kubernetes.

La mas grande diferencia es el tamaño y lugar. **Doker Compose**, sirve para que yo pueda levantar mi frontend, mi backend y mi base de datos a la vez en mi propio ordenador con un solo archivo. En cuando el **Kubernetes**, está hecho para gestionar cientos de contenedores moviéndose en muchos servidores distintos a la vez. Si un servidor se rompe, Kubernetes mueve el contenedor a otro sitio automáticamente para que la web no se caiga.

- Pon un ejemplo de uso concreto de cada uno de ellos (Docker Compose y Kubernetes).
Explica si sería posible utilizar Compose en lugar de Kubernetes y viceversa.

Docker Compose: Lo que he hecho en esta práctica. Es perfecto para desarrollar y probar mi aplicación de personajes de DC localmente antes de entregarla.

Kubernetes: Una aplicación como Instagram o Spotify, donde millones de personas entran a la vez y necesitas que el sistema crezca o se encoja solo según la gente que haya conectada.

Si se pondría utilizar Kubernetes en vez de Compose, pero es demasiado grande y complejo de montarle para un proyecto o una práctica tan pequeña, simplemente no tiene sentido.

Si podría usar Compose en lugar de Kubernetes para algo grande, pero si mi ordenador se apaga o falla, la aplicación dejaría de funcionar para todo el mundo porque Compose no sabe "repartir el trabajo" entre varias máquinas ni arreglarse solo si algo falla.